

```

1  """Lecture 7 demonstrations: FIR design, windowing, and PyTorch Conv1d.
2
3  Designed for a senior/first-year graduate DSP + ML course.
4  """
5  from __future__ import annotations
6
7  from pathlib import Path
8  import numpy as np
9  import matplotlib.pyplot as plt
10 from scipy import signal
11 import torch
12 import torch.nn.functional as F
13
14 OUT = Path(__file__).resolve().parent / "assets"
15 OUT.mkdir(exist_ok=True)
16
17
18 def ideal_lowpass(length: int, wc: float, window: str | None = None) -> np.ndarray:
19     """Return a length-N linear-phase lowpass FIR by sampling the ideal sinc.
20
21     Parameters
22     -----
23     length:
24         Number of taps. Odd length makes the symmetry center an integer.
25     wc:
26         Digital cutoff in rad/sample,  $0 < wc < \pi$ .
27     window:
28         Optional SciPy window name: boxcar, bartlett, hann, hamming, blackman.
29     """
30     if length % 2 == 0:
31         raise ValueError("Use an odd length for this centered demonstration.")
32     if not 0 < wc < np.pi:
33         raise ValueError("wc must lie between 0 and pi.")
34
35     m = (length - 1) / 2
36     n = np.arange(length)
37     r = n - m
38     hd = np.empty(length, dtype=float)
39     nonzero = r != 0
40     hd[nonzero] = np.sin(wc * r[nonzero]) / (np.pi * r[nonzero])
41     hd[~nonzero] = wc / np.pi
42
43     if window is not None:
44         w = signal.get_window(window, length, fftbins=False)
45         hd = hd * w
46     return hd
47
48
49 def plot_ideal_and_truncated() -> None:
50     wc = 0.4 * np.pi
51     n_inf = np.arange(-40, 41)
52     h_inf = np.empty_like(n_inf, dtype=float)
53     h_inf[n_inf == 0] = wc / np.pi
54     nz = n_inf != 0
55     h_inf[nz] = np.sin(wc * n_inf[nz]) / (np.pi * n_inf[nz])
56
57     fig, ax = plt.subplots(figsize=(8.5, 4.8))
58     markerline, stemlines, baseline = ax.stem(n_inf, h_inf, basefmt=" ")
59     ax.set_title("Ideal lowpass impulse response: infinite, two-sided sinc")
60     ax.set_xlabel("n")
61     ax.set_ylabel("$h_d[n]$")
62     ax.grid(True, alpha=0.25)
63     fig.tight_layout()
64     fig.savefig(OUT / "fir_ideal_sinc.png", dpi=180)
65     plt.close(fig)
66
67

```

```

68 def plot_windows() -> None:
69     N = 51
70     windows = {
71         "Rectangular": signal.windows.boxcar(N),
72         "Bartlett": signal.windows.bartlett(N),
73         "Hann": signal.windows.hann(N, sym=True),
74         "Hamming": signal.windows.hamming(N, sym=True),
75         "Blackman": signal.windows.blackman(N, sym=True),
76     }
77     fig, ax = plt.subplots(figsize=(8.8, 5.0))
78     for name, w in windows.items():
79         ax.plot(w, label=name, linewidth=1.8)
80     ax.set_title("Common windows in the time domain")
81     ax.set_xlabel("Tap index")
82     ax.set_ylabel("$w[n]$")
83     ax.legend(ncol=3)
84     ax.grid(True, alpha=0.25)
85     fig.tight_layout()
86     fig.savefig(OUT / "fir_windows_time.png", dpi=180)
87     plt.close(fig)
88
89
90 def plot_windowed_responses() -> None:
91     wc = 0.4 * np.pi
92     N = 51
93     names = ["boxcar", "bartlett", "hann", "hamming", "blackman"]
94     labels = ["Rectangular", "Bartlett", "Hann", "Hamming", "Blackman"]
95
96     fig, ax = plt.subplots(figsize=(8.8, 5.0))
97     for name, label in zip(names, labels):
98         h = ideal_lowpass(N, wc, name)
99         w, H = signal.freqz(h, worN=4096)
100         Hdb = 20 * np.log10(np.maximum(np.abs(H), 1e-8))
101         ax.plot(w / np.pi, Hdb, label=label, linewidth=1.6)
102     ax.axvline(wc / np.pi, linestyle="--", linewidth=1.2)
103     ax.set_ylim(-110, 5)
104     ax.set_xlim(0, 1)
105     ax.set_title("Window choice trades transition width against stopband ripple")
106     ax.set_xlabel(r"Normalized frequency $\omega/\pi$")
107     ax.set_ylabel("Magnitude (dB)")
108     ax.grid(True, alpha=0.25)
109     ax.legend(ncol=2)
110     fig.tight_layout()
111     fig.savefig(OUT / "fir_window_responses.png", dpi=180)
112     plt.close(fig)
113
114
115 def plot_gibbs_zoom() -> None:
116     wc = 0.4 * np.pi
117     fig, ax = plt.subplots(figsize=(8.8, 5.0))
118     for N in [21, 51, 101]:
119         h = ideal_lowpass(N, wc, "boxcar")
120         w, H = signal.freqz(h, worN=8192)
121         ax.plot(w / np.pi, np.abs(H), label=f"N={N}")
122     ax.axvline(wc / np.pi, linestyle="--", linewidth=1.2)
123     ax.set_xlim(0.25, 0.55)
124     ax.set_ylim(-0.12, 1.15)
125     ax.set_title("Rectangular truncation: the ripple narrows, but the overshoot persists")
126
127     ax.set_xlabel(r"Normalized frequency $\omega/\pi$")
128     ax.set_ylabel(r"$|H(e^{j\omega})|$")
129     ax.grid(True, alpha=0.25)
130     ax.legend()
131     fig.tight_layout()
132     fig.savefig(OUT / "fir_gibbs_zoom.png", dpi=180)
133     plt.close(fig)

```

```

134
135 def plot_phase_group_delay() -> None:
136     N = 41
137     h = ideal_lowpass(N, 0.35 * np.pi, "hamming")
138     w, H = signal.freqz(h, worN=2048)
139     phase = np.unwrap(np.angle(H))
140     gd_w, gd = signal.group_delay((h, 1), w=1024)
141
142     fig, ax = plt.subplots(figsize=(8.8, 5.0))
143     ax.plot(w / np.pi, phase, linewidth=1.8)
144     ax.set_title("Symmetric FIR: approximately linear phase in the passband")
145     ax.set_xlabel(r"Normalized frequency  $\omega/\pi$ ")
146     ax.set_ylabel("Unwrapped phase (rad)")
147     ax.grid(True, alpha=0.25)
148     fig.tight_layout()
149     fig.savefig(OUT / "fir_linear_phase.png", dpi=180)
150     plt.close(fig)
151
152     fig, ax = plt.subplots(figsize=(8.8, 5.0))
153     ax.plot(gd_w / np.pi, gd, linewidth=1.8)
154     ax.axhline((N - 1) / 2, linestyle="--", linewidth=1.2,
155               label=f"${(N-1)/2}={ (N-1)/2:g} $ samples")
156     ax.set_xlim(0, 0.7)
157     ax.set_ylim(0, N)
158     ax.set_title("Linear phase implies nearly constant group delay")
159     ax.set_xlabel(r"Normalized frequency  $\omega/\pi$ ")
160     ax.set_ylabel("Group delay (samples)")
161     ax.grid(True, alpha=0.25)
162     ax.legend()
163     fig.tight_layout()
164     fig.savefig(OUT / "fir_group_delay.png", dpi=180)
165     plt.close(fig)
166
167
168 def torch_fixed_fir_demo() -> tuple[float, float]:
169     rng = np.random.default_rng(7)
170     fs = 8000
171     t = np.arange(0, 0.04, 1 / fs)
172     x = (np.sin(2 * np.pi * 450 * t)
173          + 0.45 * np.sin(2 * np.pi * 2100 * t)
174          + 0.05 * rng.standard_normal(t.size)).astype(np.float32)
175
176     h = ideal_lowpass(31, 0.30 * np.pi, "hamming").astype(np.float32)
177     y_np = np.convolve(x, h, mode="full")[:len(x)]
178
179     xt = torch.from_numpy(x).view(1, 1, -1)
180     ht = torch.from_numpy(h)
181     kernel = torch.flip(ht, dims=[0]).view(1, 1, -1)
182     xpad = F.pad(xt, (len(h) - 1, 0))
183     y_t = F.conv1d(xpad, kernel).view(-1).numpy()
184
185     max_error = float(np.max(np.abs(y_np - y_t)))
186
187     fig, ax = plt.subplots(figsize=(8.8, 5.0))
188     nshow = 150
189     ax.plot(x[:nshow], label="input", linewidth=1.1)
190     ax.plot(y_np[:nshow], label="NumPy DSP convolution", linewidth=1.8)
191     ax.plot(y_t[:nshow], "--", label="PyTorch Conv1d result", linewidth=1.4)
192     ax.set_title("A fixed FIR can be implemented exactly with PyTorch Conv1d")
193     ax.set_xlabel("Sample index")
194     ax.set_ylabel("Amplitude")
195     ax.grid(True, alpha=0.25)
196     ax.legend()
197     fig.tight_layout()
198     fig.savefig(OUT / "fir_torch_match.png", dpi=180)
199     plt.close(fig)
200

```

```
201     # Demonstrate fixed versus learnable weights.
202     weights = torch.nn.Parameter(kernel.clone())
203     grad_flag = float(weights.requires_grad)
204     return max_error, grad_flag
205
206
207 def main() -> None:
208     plot_ideal_and_truncated()
209     plot_windows()
210     plot_windowed_responses()
211     plot_gibbs_zoom()
212     plot_phase_group_delay()
213     max_error, grad_flag = torch_fixed_fir_demo()
214     print(f"PyTorch/NumPy max convolution mismatch: {max_error:.3e}")
215     print(f"Example learnable kernel requires_grad: {bool(grad_flag)}")
216
217
218 if __name__ == "__main__":
219     main()
220
```